Modèles Algorithmiques Fondamentaux : Parcours et Recherche

Thème principal : Initiation au développement et conception algorithmique

Synthèse exécutive : Ce cours pose le socle méthodologique fondamental de la conception algorithmique en BUT 1 Informatique, indispensable pour la réussite des partiels et des SAE. Avant toute écriture de code, l'étudiant doit analyser le contrat d'interface et le mécanisme d'accès de sa collection de données (Direct, Séquentiel ou Dynamique). Le cours détaille trois modèles canoniques réutilisables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence. Une rigueur absolue est exigée sur la gestion des bornes d'indices (de 0 à N-1) pour éviter les débordements de mémoire ('out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer au minimum deux conditions d'arrêt pour tout algorithme de recherche.

Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape zéro de toute démarche d'analyse algorithmique consiste à s'interroger sur la nature des accès autorisés par la structure de données sous-jacente. Si l'on manipule une structure à accès direct (un tableau statique de N éléments ou une chaîne de caractères), l'adresse mémoire de n'importe quel élément situé à l'indice 'i' est calculable immédiatement. En revanche, sur un flux séquentiel (tel qu'un fichier CSV lu ligne par ligne), l'ordinateur ne propose que trois opérations fondamentales : se positionner au début du flux, lire l'élément suivant, et tester si la fin du flux est atteinte. Il est physiquement impossible de sauter directement à la 50ème ligne sans lire les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue un contre-sens algorithmique qui invalide entièrement la solution proposée.

- Structure de données : Collection organisée regroupant des éléments de même type.
- Accès Direct : Capacité matérielle ou logique d'atteindre instantanément n'importe quel élément via son indice 'i' en temps constant O(1).
- Accès Séquentiel : Contrainte imposant de démarrer au début et de progresser pas à pas (début, suivant, fin) sans saut direct (ex: fichiers CSV).

Cas concret enseignant : *Piège récurrent en évaluation souligné par l'enseignant : rédiger un algorithme qui utilise un accès par indice 'T[i]' pour traiter un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne fournit pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à la note zéro.*

Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une zone définie d'une collection doit subir une opération uniforme sans aucune exception ni filtrage (exemples : initialiser toutes les cases d'un tableau à zéro, ou afficher le contenu complet). Dans un tableau de N cases en convention standard (indexation à zéro), la première case se situe à l'indice 0 et la dernière à l'indice N - 1. Le traitement est exécuté de manière indifférenciée à chaque tour de boucle.

- Exhaustivité : 100% des éléments de la zone délimitée subissent le traitement.
- Itération bornée : Utilisation obligatoire d'une boucle 'Pour' (For) car le nombre d'itérations est calculable à l'avance.
- Erreur Out of Bounds : Tenter d'accéder à l'indice N dans un tableau de taille N provoque un crash mémoire.

Cas concret enseignant : *Exemple d'illustration vu en amphithéâtre : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case mémoire T[N] qui n'existe pas.*

Second Modèle : Le Parcours Complet avec Traitement Conditionné

Le parcours complet avec traitement conditionné conserve l'inspection exhaustive du modèle précédent, mais encadre l'action de traitement par une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter les voyelles), afficher uniquement un sous-ensemble (ex: nombres pairs), ou effectuer des calculs cumulatifs ciblés. Si la condition compare l'élément courant à ses voisins (i-1 ou i+1), les bornes de la boucle doivent être ajustées pour éviter d'évaluer des indices hors limites.

- Parcours exhaustif : Tous les éléments de la zone sont inspectés de la borne initiale à la borne finale.
- Traitement sélectif : L'action principale n'est déclenchée que si l'élément inspecté valide un prédicat logique.
- Architecture : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection), mais l'inspecteur n'appose le tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À l'inverse des parcours complets, la recherche de première occurrence s'interrompt dès que le premier élément satisfaisant le critère est localisé. L'esprit de conception repose sur le pessimisme : on initialise un booléen 'trouve' à 'Faux' avant d'engager l'exploration. La Variante 1 (Rupture immédiate) sort de la boucle dès le succès et laisse l'indice 'i' positionné exactement sur l'élément trouvé. La Variante 2 (Tant que global) utilise une condition unique en tête ('Tant que i < N et NON trouve'), plus simple à coder si seule l'existence booléenne importe, mais nécessite un réajustement de l'indice en sortie.

- Pessimisme algorithmique : Postulat de départ supposant que la cible est absente de la collection (trouve <- Faux).
- Itération non bornée : Emploi obligatoire d'une boucle 'Tant que' (While) car le nombre d'itérations est imprévisible.
- Double condition d'arrêt : Arrêt si l'élément est trouvé (succès) OU si la collection est entièrement parcourue (échec).

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le modèle du parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, du moment que la structure dynamique propose un mécanisme d'indexation ou un itérateur permettant de parcourir l'ensemble des éléments de manière séquentielle ou directe sans rupture de contrat.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que le choix dépend de votre besoin après la boucle : si vous avez besoin de savoir exactement où se trouve l'élément (son indice), la variante 1 est indispensable. Si vous voulez juste savoir s'il existe (vrai/faux), la variante 2 est plus compacte et évite les erreurs de gestion de rupture.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionné dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement décaler la borne de début de votre boucle à i = 1 au lieu de i = 0. Si vous commencez à 0, l'évaluation de T[-1] provoquera un accès mémoire interdit.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser un modèle de recherche de première occurrence (qui s'arrête au premier), mais un modèle de parcours complet avec traitement conditionné, où l'action est d'enregistrer ou d'afficher l'indice i à chaque fois que T[i] = 5.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments.

Résolution pas à pas :
Étape 1 : Analyse du problème. Il s'agit d'inspecter l'intégralité d'un tableau (accès direct) et de filtrer l'affichage selon un critère (> 0). Le modèle retenu est le parcours complet avec traitement conditionné. Étape 2 : Bornes d'indices. Pour un tableau de N éléments indicé à zéro, les bornes vont de 0 à N - 1. Étape 3 : Structure de contrôle. Le nombre d'itérations étant connu, on utilise une boucle 'Pour i allant de 0 à N - 1'. Étape 4 : Filtre et action. À chaque itération, évaluer 'Si T[i] > 0 Alors Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on impérativement utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : Le modèle de recherche de première occurrence (avec boucle Tant que et double condition d'arrêt).

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce que l'indexation commence à 0 en informatique standard, ce qui signifie que la nième et dernière case se trouve à l'indice N - 1. Accéder à l'indice N provoque un dépassement de mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence et quelles sont-elles ?

R : Exactement deux conditions d'arrêt : le succès de la recherche (élément trouvé) et l'échec par épuisement de la collection (fin de tableau atteinte).

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765